JUnit 5 Unit Testing Handout

1 What is a Unit Test?

A unit test checks one small piece of code (usually a single method) in isolation.[1] In Java, we commonly use the JUnit 5 testing framework to write unit tests.[2]

- Goal: verify that one class or method behaves correctly in many scenarios.
- Tests should be fast, repeatable, and not depend on external systems (DBs, network, etc.).

Test classes usually live under `src/test/java` and are named with the suffix `Test`, for example `CalculatorTest`.

2 Key JUnit 5 Annotations

- **`@Test`**: marks a method as a test.[3]
- **`@BeforeEach`**: runs before each test (good for setting up fresh objects).
- **`@BeforeAll`** / **`@AfterAll`**: run once before/after all tests in a class.
- **`@DisplayName`**: provides a human-readable name for a test in reports.

3 Common JUnit Assertions

Most assertions come from `org.junit.jupiter.api.Assertions` and are often statically imported.[4]

- `assertEquals(expected, actual)`: Asserts that expected and actual are equal.
- `assertNotEquals(unexpected, actual)`: Asserts that expected and actual are not equal.
- `assertTrue(condition)`, `assertFalse(condition)`: Asserts that the supplied condition is true for former, false for latter.
- `assertNull(obj)`, `assertNotNull(obj)`: Asserts that actual is null or not null, accordingly.

[1] See e.g. JUnit 5 introductions for basics and motivation [3, 5].
[2] Overview of JUnit 5 features and setup [1, 2].
[3] Basic test annotation usage in JUnit 5 [1, 3].
[4] See overviews of JUnit 5 assertion methods [4, 6].

- `assertThrows(ExceptionType.class, () -> code)`: Asserts that execution of the supplied executable throws an exception of the expectedType and returns the exception.

For more information: `https://docs.junit.org/5.0.1/api/org/junit/jupiter/api/Assertions.html`

Example exception test:

```
@Test
void divide_byZero_throwsException() {
    Calculator calculator = new Calculator();

    IllegalArgumentException ex = assertThrows(
            IllegalArgumentException.class,
            () -> calculator.divide(10, 0)
    );
    assertEquals("b must not be zero", ex.getMessage());
}
```

4 Good Unit Test Habits

- One clear scenario per test method; avoid huge multi-purpose tests.[5]
- Use descriptive test names that encode method, condition, and expected result (e.g. `withdraw_insufficie`
- Keep tests simple and readable; minimize logic (few `if`s or loops).

A common pattern is **Arrange–Act–Assert**:

```
@Test
void add_twoPositiveNumbers_returnsSum() {
    // Arrange
    Calculator calculator = new Calculator();

    // Act
    int result = calculator.add(2, 3);

    // Assert
    assertEquals(5, result);
}
```

5 Using Mockito with JUnit 5

Mockito is a mocking framework that lets you replace dependencies with mock objects so you can test a class in isolation.[6]

[5] Typical Java unit testing best-practice lists [8, 7].

[6] High-level guides on using Mockito with JUnit 5 [9, 10].

Dependencies (Maven example)

```
<dependencies>
    <!-- JUnit 5 -->
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>5.10.0</version>
        <scope>test</scope>
    </dependency>

    <!-- Mockito core -->
    <dependency>
        <groupId>org.mockito</groupId>
        <artifactId>mockito-core</artifactId>
        <version>5.11.0</version>
        <scope>test</scope>
    </dependency>

    <!-- Mockito + JUnit 5 integration -->
    <dependency>
        <groupId>org.mockito</groupId>
        <artifactId>mockito-junit-jupiter</artifactId>
        <version>5.11.0</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

This adds the Mockito JUnit Jupiter extension so mocks can be initialized automatically in JUnit 5 tests.[7]

Example: Service with Mocked Repository

Production code:

```
public interface UserRepository {
    User findById(long id);
    void save(User user);
}

public class UserService {
    private final UserRepository repo;

    public UserService(UserRepository repo) {
        this.repo = repo;
    }

    public User loadUser(long id) {
        return repo.findById(id);
    }

    public void registerUser(User user) {
        // some domain logic...
        repo.save(user);
```

[7] See the Mockito JUnit Jupiter extension documentation and examples [12, 11].

```
    }
}
```

JUnit 5 test with Mockito:

```
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class UserServiceTest {

    @Mock
    private UserRepository userRepository;

    @InjectMocks
    private UserService userService;

    @Test
    void loadUser_returnsUserFromRepository() {
        User user = new User(1L, "Alice");
        when(userRepository.findById(1L)).thenReturn(user);

        User result = userService.loadUser(1L);

        assertEquals("Alice", result.getName());
        verify(userRepository, times(1)).findById(1L);
        verifyNoMoreInteractions(userRepository);
    }

    @Test
    void registerUser_savesUserViaRepository() {
        User user = new User(2L, "Bob");

        userService.registerUser(user);

        verify(userRepository).save(user);
    }
}
```

Key ideas:

- `@ExtendWith(MockitoExtension.class)` connects Mockito with JUnit 5.[8]
- `@Mock` marks dependencies to be mocked.
- `@InjectMocks` creates the class under test and injects the mocks.
- `when(...).thenReturn(...)` sets up stub behavior; `verify(...)` checks calls.

[8] Typical usage of the Mockito extension and annotations [10, **?**].

6 JUnit 5 Parameterized Tests

Parameterized tests run the same test method multiple times with different inputs using `@ParameterizedTest` and source annotations such as `@ValueSource` or `@CsvSource`.[9]

Single-Argument Example with `@ValueSource`

```
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;

import static org.junit.jupiter.api.Assertions.assertTrue;

class NumberUtilsTest {

    @ParameterizedTest
    @ValueSource(ints = {1, 3, 5, -3, 15, Integer.MAX_VALUE})
    void isOdd_returnsTrueForOddNumbers(int number) {
        assertTrue(NumberUtils.isOdd(number));
    }
}
```

String Example with `@ValueSource`

```
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;

import static org.junit.jupiter.api.Assertions.assertTrue;

class StringUtilsTest {

    @ParameterizedTest
    @ValueSource(strings = {"racecar", "radar", "able was I ere I saw elba"})
    void palindromes_areRecognized(String candidate) {
        assertTrue(StringUtils.isPalindrome(candidate));
    }
}
```

Multiple Arguments with `@CsvSource`

```
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;

import static org.junit.jupiter.api.Assertions.assertEquals;

class TaxCalculatorTest {

    @ParameterizedTest
    @CsvSource({
        "100.0, 0.10, 10.0",
        "200.0, 0.20, 40.0",
        "50.0,  0.00, 0.0"
    })
```

[9] General overviews of parameterized tests in JUnit 5 [13, 14, 15].

```
    void calculateTax_returnsExpected(double amount, double rate, double
        expectedTax) {
        double actual = TaxCalculator.calculateTax(amount, rate);
        assertEquals(expectedTax, actual, 0.0001);
    }
}
```

`@EnumSource` and `@MethodSource`

```
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.EnumSource;

import java.time.Month;

import static org.junit.jupiter.api.Assertions.assertTrue;

class MonthTest {

    @ParameterizedTest
    @EnumSource(Month.class)
    void monthNumber_isBetweenOneAndTwelve(Month month) {
        int value = month.getValue();
        assertTrue(value >= 1 && value <= 12);
    }
}
```

```
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.MethodSource;

import java.util.stream.Stream;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.params.provider.Arguments.arguments;
import org.junit.jupiter.params.provider.Arguments;

class StringBlankTest {

    static Stream<Arguments> blankStrings() {
        return Stream.of(
            arguments(null, true),
            arguments("", true),
            arguments(" ", true),
            arguments("not blank", false)
        );
    }

    @ParameterizedTest
    @MethodSource("blankStrings")
    void isBlank_handlesNullAndWhitespace(String input, boolean expected) {
        boolean result = StringUtils.isBlank(input);
        assertEquals(expected, result);
    }
}
```

7 Student Exercise:

- Given a small class (e.g. `BankAccount`), ask students to:
  - Identify behaviors (deposit, withdraw, overdraft, etc.).
  - Write one unit test per behavior, including error cases.
- Refactor three or four similar tests into a single parameterized test using `@ValueSource` or `@CsvSource`.
- Introduce a service with a dependency (e.g. repository) and have students mock the dependency using Mockito and verify interactions.

References

[1] Baeldung. A Guide to JUnit 5. https://www.baeldung.com/junit-5.

[2] Vogella. JUnit 5 tutorial - Learn how to write unit tests. https://www.vogella.com/tutorials/JUnit/article.html.

[3] GeeksforGeeks. Introduction to JUnit 5. https://www.geeksforgeeks.org/software-testing/introduction-to-junit-5/.

[4] GeeksforGeeks. JUnit 5 - Assertions. https://www.geeksforgeeks.org/software-testing/junit-5-assertions/.

[5] DigitalOcean. JUnit5 Tutorial. https://www.digitalocean.com/community/tutorials/junit5-tutorial.

[6] Baeldung. Assertions in JUnit 4 and JUnit 5. https://www.baeldung.com/junit-assertions.

[7] Code Intelligence. 11 Tips for Unit Testing in Java. https://www.code-intelligence.com/blog/11-tips-unit-testing-java.

[8] Baeldung. Best Practices for Unit Testing in Java. https://www.baeldung.com/java-unit-testing-best-practices.

[9] BrowserStack. How to use Mockito with JUnit 5: A Beginner's Guide. https://www.browserstack.com/guide/junit-5-mockito.

[10] A. Huttunen. Using Mockito With JUnit 5. https://www.arhohuttunen.com/junit-5-mockito/.

[11] Stack Overflow. How to use Mockito with JUnit 5? https://stackoverflow.com/questions/40961057/how-to-use-mockito-with-junit-5.

[12] Mockito. MockitoExtension (junit-jupiter API). https://www.javadoc.io/doc/org.mockito/mockito-junit-jupiter/latest/org/mockito/junit/jupiter/MockitoExtension.html.

[13] Baeldung. Guide to JUnit 5 Parameterized Tests. https://www.baeldung.com/parameterized-tests-junit-5.

[14] Reflectoring. JUnit 5 Parameterized Tests. https://reflectoring.io/tutorial-junit5-parameterized-tests/.

[15] GeeksforGeeks. JUnit 5 - How to Write Parameterized Tests. https://www.geeksforgeeks.org/software-testing/junit-5-how-to-write-parameterized-tests/.